

api_post_data.php

```
<?php
require_once __DIR__ . '/config.php';

header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agriculture')
header('Access-Control-Allow-Credentials: true');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: POST, GET, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}

session_start();
header('Content-Type: application/json');

$input = json_decode(file_get_contents('php://input'), true);
if (!$input || !isset($input['device_id'])) {
    http_response_code(400);
    echo json_encode(['error' => 'Missing or invalid JSON body']);
    exit;
}

try {
    $db = get_db();
    $stmt = $db->prepare("INSERT INTO sensor_data
        (device_id, temperature, humidity, water_level, water_drop, status, min_temp, max_t
min_hum, max_hum)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)");
    $stmt->execute([
        $input['device_id'],
        $input['temperature'] ?? null,
        $input['humidity'] ?? null,
        $input['water_level'] ?? null,
        $input['water_drop'] ?? 0,
        $input['status'] ?? 'Normal',
        $input['min_temp'] ?? null,
        $input['max_temp'] ?? null,
        $input['min_hum'] ?? null,
        $input['max_hum'] ?? null,
    ]);
    echo json_encode(['success' => true, 'id' => $db->lastInsertId()]);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(['error' => $e->getMessage()]);
}
```

Api_get_settings.php

```
<?php
require_once __DIR__ . '/config.php';
header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agricultu');
header('Access-Control-Allow-Credentials: true');
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: POST, GET, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}
session_start();
header('Content-Type: application/json');
$device = $_GET['device_id'] ?? 'esp32_01';
try {
    $db = get_db();
    $stmt = $db->prepare("SELECT * FROM device_settings WHERE device_id = ?");
    $stmt->execute([$device]);
    $settings = $stmt->fetch();
    echo json_encode($settings ?: ['error' => 'Device not found']);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(['error' => $e->getMessage()]);
}
```

Api_update_settings.php

```
<?php
require_once __DIR__ . '/config.php';

header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agriculture/');
header('Access-Control-Allow-Credentials: true');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: POST, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Content-Type: application/json');
    http_response_code(405);
    echo json_encode(['error' => 'Method Not Allowed']);
    exit;
}

session_start();
require_auth();

header('Content-Type: application/json');

$input = json_decode(file_get_contents('php://input'), true);
if (!$input) {
    http_response_code(400);
    echo json_encode(['error' => 'Invalid JSON']);
    exit;
}
```

```

try {
    $db = get_db();
    $stmt = $db->prepare("UPDATE device_settings SET
        temp_threshold      = ?,
        humidity_threshold  = ?,
        water_level_threshold = ?,
        upload_interval     = ?,
        buzzer_enabled      = ?,
        servo1_open         = ?,
        servo2_open         = ?,
        updated_at          = CURRENT_TIMESTAMP
        WHERE device_id = ?");
    $stmt->execute([
        $input['temp_threshold']      ?? 31.0,
        $input['humidity_threshold']  ?? 70.0,
        $input['water_level_threshold'] ?? 26.5,
        $input['upload_interval']     ?? 10,
        $input['buzzer_enabled']      ?? 1,
        $input['servo1_open']         ?? 0,
        $input['servo2_open']         ?? 0,
        $input['device_id']           ?? 'esp32_01',
    ]);
    echo json_encode(['success' => true]);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(['error' => $e->getMessage()]);
}

```

Api_stream.php

```

<?php
require_once __DIR__ . '/config.php';
header('Content-Type: text/event-stream');
header('Cache-Control: no-cache');
header('Connection: keep-alive');
header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agricultu');
header('Access-Control-Allow-Credentials: true');
//CRITICAL: disable all PHP output buffering
//Server-Sent Events (SSE) real time 1 second change
if (function_exists('apache_setenv')) @apache_setenv('no-gzip', '1');
@ini_set('zlib.output_compression', '0');
@ini_set('output_buffering', '0');
@ini_set('implicit_flush', '1');
while (ob_get_level()) ob_end_flush();
ob_implicit_flush(true);
set_time_limit(0);

$db = get_db();
$lastId = 0;
while (true) {
    try { // get all data
        $stmt = $db->query("SELECT * FROM sensor_data ORDER BY id DESC LIMIT 1");
        $row = $stmt->fetch();

        if ($row && $row['id'] != $lastId) {
            $lastId = $row['id'];
            echo "data: " . json_encode($row) . "\n\n";
            flush(); //push immediately to browser
        }
    } catch (PDOException $e) {
        echo "data: " . json_encode(['error' => $e->getMessage()]) . "\n\n";
    }
}

```

```

        flush();
    }
    // Keep-alive comment so the browser doesn't drop the connection
    echo ": ping\n\n";
    flush();
    sleep(1);    // polls database every 1 second
}

```

Api_get_history.php

```

<?php
require_once __DIR__ . '/config.php';

session_start();
header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agricultu');
header('Access-Control-Allow-Credentials: true');
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: POST, GET, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}
require_auth();
header('Content-Type: application/json');
try {
    $db = get_db();
    $device = $_GET['device_id'] ?? 'esp32_01';
    $minutes = (int)($_GET['minutes'] ?? 30);    // default: last 30 minutes
    // First try: get data from last N minutes (real-time)
    $stmt = $db->prepare("
        SELECT * FROM sensor_data
        WHERE device_id = ?
        AND created_at >= DATE_SUB(NOW(), INTERVAL {$minutes} MINUTE)
        ORDER BY id ASC
    ");
    $stmt->execute([$device]);
    $data = $stmt->fetchAll();
    // Fallback: if no recent data, get last 30 records
    if (empty($data)) {
        $stmt = $db->prepare("
            SELECT * FROM sensor_data
            WHERE device_id = ?
            ORDER BY id DESC
            LIMIT 30
        ");
        $stmt->execute([$device]);
        $data = array_reverse($stmt->fetchAll());
    }
    echo json_encode($data);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(['error' => $e->getMessage()]);
}

```

Api_get_latest.php

```

<?php
require_once __DIR__ . '/config.php';
header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agricultu
header('Access-Control-Allow-Credentials: true');
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: POST, GET, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}
session_start();
require_auth();
header('Content-Type: application/json');
try {
    $db = get_db();
    $stmt = $db->query("SELECT * FROM sensor_data ORDER BY id DESC LIMIT 1");
    $data = $stmt->fetch();
    echo json_encode($data ?: ['error' => 'No data yet']);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(['error' => $e->getMessage()]);
}

```

Api_check_auth.php

```

<?php
require_once __DIR__ . '/config.php';

session_start(); // once, before any output

header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agricultu
header('Access-Control-Allow-Credentials: true');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: GET, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}

header('Content-Type: application/json');

if (!empty($_SESSION['authenticated']) && $_SESSION['authenticated'] === true) {
    echo json_encode(['authenticated' => true]);
} else {
    http_response_code(401);
    echo json_encode(['authenticated' => false]);
}

```

Api_login.php

```

<?php
require_once __DIR__ . '/config.php';
header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agriculture');
header('Access-Control-Allow-Credentials: true');
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: POST, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}
session_start();
header('Content-Type: application/json');
$input = json_decode(file_get_contents('php://input'), true);
if (!$input || empty($input['username']) || empty($input['password'])) {
    http_response_code(400);
    echo json_encode(['error' => 'Missing username or password']);
    exit;
}
if ($input['username'] === AUTH_USER && password_verify($input['password'], AUTH_PASS_HASH)) {
    $_SESSION['authenticated'] = true;
    $_SESSION['login_time'] = time();
    echo json_encode(['success' => true]);
} else {
    http_response_code(401);
    echo json_encode(['error' => 'Invalid credentials']);
}
}

```

Api_logout.php

```

<?php
require_once __DIR__ . '/config.php';

header('Access-Control-Allow-Origin: https://socstudentmusicforlife.com/fish_agriculture');
header('Access-Control-Allow-Credentials: true');
session_start();
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: POST, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}
header('Content-Type: application/json');
$_SESSION = [];
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}
session_destroy();
echo json_encode(['success' => true]);

```

Config.php

```

<?php
/**
 * MySQL + Auth Configuration
 */

define('DB_HOST', 'localhost');
define('DB_NAME', 'musicbvk_fish_agriculture'); // database
define('DB_USER', 'musicbvk_azri'); //user change
define('DB_PASS', 'lme?;3KZm9Vv'); // authority
define('APP_FOLDER', '/fish_agriculture');

define('AUTH_USER', 'admin');
define('AUTH_PASS_HASH', '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi');
"password"

function get_db() {
    static $pdo = null;
    if ($pdo === null) {
        $dsn = sprintf('mysql:host=%s;dbname=%s;charset=utf8mb4', DB_HOST, DB_NAME);
        $pdo = new PDO($dsn, DB_USER, DB_PASS);
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
        $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, true);
    }
    return $pdo;
}

function require_auth() {
    if (empty($_SESSION['authenticated']) || $_SESSION['authenticated'] !== true) {
        http_response_code(401);
        header('Content-Type: application/json');
        echo json_encode(['error' => 'Unauthorized']);
        exit;
    }
}

```

Database.sql

```

-- SmartContainer Database Schema (with min/max tracking)

CREATE TABLE IF NOT EXISTS `sensor_data` (
    `id` INT AUTO_INCREMENT PRIMARY KEY,
    `device_id` VARCHAR(32) NOT NULL,
    `temperature` FLOAT,
    `humidity` FLOAT,
    `water_level` INT,
    `water_drop` TINYINT DEFAULT 0,
    `status` VARCHAR(20) DEFAULT 'Normal',
    `min_temp` DECIMAL(5,2) NULL,
    `max_temp` DECIMAL(5,2) NULL,
    `min_hum` DECIMAL(5,2) NULL,
    `max_hum` DECIMAL(5,2) NULL,
    `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `device_settings` (
    `id` INT PRIMARY KEY,
    `device_id` VARCHAR(32) NOT NULL,
    `temp_threshold` FLOAT DEFAULT 31.0,
    `humidity_threshold` FLOAT DEFAULT 60.0,
    `water_level_threshold` INT DEFAULT 500,
    `upload_interval` INT DEFAULT 10,

```

```

        `buzzer_enabled`          TINYINT DEFAULT 1,
        `servo1_open`             TINYINT DEFAULT 0,
        `servo2_open`             TINYINT DEFAULT 0,
        `updated_at`              DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

INSERT INTO `device_settings` (`id`, `device_id`)
SELECT 1, 'esp32_01'
WHERE NOT EXISTS (SELECT 1 FROM `device_settings` WHERE `id` = 1);

```

Index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Smart Agriculture · IoT Dashboard</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link
href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500;600&family=Inter@300;400;500;600;700&display=swap" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/chart.js/4.4.0/chart.umd.min.js"></script>
<style>
:root {
  --bg: #151d2e; --surface: #1c2540; --surface2: #243058; --border: #2e3f6a; --border-lo:
#253250;
  --accent: #5b9cf6; --green: #36c98e; --yellow: #f5a94a; --red: #f06252; --purple: #a78bfa;
  --c-temp: #f06252; --c-hum: #5b9cf6; --c-water: #36c98e; --c-rain: #a78bfa;
  --text: #dde5f8; --muted: #7a8fb8;
  --mono: 'JetBrains Mono', monospace; --sans: 'Inter', sans-serif;
  --radius: 12px; --radius-sm: 8px; --shadow: 0 4px 20px rgba(0,0,0,.35);
}
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
body {
  background: var(--bg); color: var(--text); font-family: var(--sans); font-size: 14px;
  line-height: 1.6; min-height: 100vh;
  background-image: radial-gradient(ellipse 60% 40% at 70% 0%, rgba(91,156,246,.07) 0%,
transparent 70%), radial-gradient(ellipse 40% 30% at 0% 80%, rgba(54,201,142,.05) 0%,
transparent 60%);
}
.shell { max-width: 1260px; margin: 0 auto; padding: 28px 24px 72px; }
header { display: flex; align-items: center; gap: 14px; margin-bottom: 28px; padding-bottom:
22px; border-bottom: 1px solid var(--border-lo); }
.logo { width: 42px; height: 42px; background: linear-gradient(135deg, var(--accent) 0%,
#2563eb 100%); border-radius: 10px; display: grid; place-items: center; font-size: 20px;
box-shadow: 0 4px 12px rgba(91,156,246,.3); flex-shrink: 0; }
.header-text h1 { font-size: 17px; font-weight: 700; letter-spacing: -0.4px; color:
var(--text); }
.header-text p { color: var(--muted); font-size: 11px; font-family: var(--mono); margin-top:
1px; }
.header-right { margin-left: auto; display: flex; align-items: center; gap: 10px; }
.badge { display: flex; align-items: center; gap: 7px; font-family: var(--mono); font-size:
11px; color: var(--muted); background: var(--surface); border: 1px solid var(--border-lo);
padding: 5px 12px; border-radius: 20px; }
.badge .dot { width: 7px; height: 7px; border-radius: 50%; background: var(--green); animation:
pulse 2s ease infinite; }
.badge .dot.offline { background: var(--red); animation: none; }
@keyframes pulse { 0%,100% { opacity: 1; box-shadow: 0 0 0 0 rgba(54,201,142,.4); } 50% {
opacity: .5; box-shadow: 0 0 0 4px rgba(54,201,142,0); } }

```



```

#statusBar { display: flex; align-items: center; gap: 10px; padding: 11px 18px; border-radius: var(--radius-sm); font-family: var(--mono); font-size: 12px; margin-bottom: 22px; transition: all 0.3s; }
#statusBar.normal { background: rgba(54,201,142,.08); border: 1px solid rgba(54,201,142,.2); color: var(--green); }
#statusBar.warning { background: rgba(245,169,74,.08); border: 1px solid rgba(245,169,74,.2); color: var(--yellow); }
#statusBar.danger { background: rgba(240,98,82,.08); border: 1px solid rgba(240,98,82,.2); color: var(--red); }
#statusBar.stale { background: rgba(245,169,74,.08); border: 1px solid rgba(245,169,74,.2); color: var(--yellow); }
.status-icon { font-size: 15px; }
.grid-4 { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 10px; }
.grid-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-bottom: 20px; }
.grid-1 { margin-bottom: 20px; }
@media (max-width: 960px) { .grid-4 { grid-template-columns: 1fr 1fr; } }
@media (max-width: 640px) { .grid-4, .grid-2 { grid-template-columns: 1fr; } }
.card { background: var(--surface); border: 1px solid var(--border-lo); border-radius: var(--radius); padding: 20px 22px; box-shadow: var(--shadow); }
.sensor-card { position: relative; overflow: hidden; padding-left: 26px; }
.sensor-card::before { content: ''; position: absolute; left: 0; top: 0; bottom: 0; width: border-radius: 4px 0 0 4px; }
.sensor-card.temp::before { background: var(--c-temp); }
.sensor-card.hum::before { background: var(--c-hum); }
.sensor-card.water::before { background: var(--c-water); }
.sensor-card.rain::before { background: var(--c-rain); }
.sensor-card .sensor-icon { width: 32px; height: 32px; border-radius: 8px; display: grid; place-items: center; font-size: 16px; margin-bottom: 10px; }
.sensor-card.temp .sensor-icon { background: rgba(240,98,82,.12); }
.sensor-card.hum .sensor-icon { background: rgba(91,156,246,.12); }
.sensor-card.water .sensor-icon { background: rgba(54,201,142,.12); }
.sensor-card.rain .sensor-icon { background: rgba(167,139,250,.12); }
.card-label { font-size: 10px; text-transform: uppercase; letter-spacing: 0.1em; color: var(--muted); margin-bottom: 6px; font-family: var(--mono); font-weight: 500; }
.card-value { font-family: var(--mono); font-size: 34px; font-weight: 600; line-height: 1; color: var(--text); letter-spacing: -1px; }
.card-unit { font-size: 15px; color: var(--muted); margin-left: 3px; font-weight: 400; }
.card-sub { margin-top: 9px; font-size: 11px; color: var(--muted); font-family: var(--mono); line-height: 1.4; }
.sensor-card.temp .card-value { color: var(--c-temp); }
.sensor-card.hum .card-value { color: var(--c-hum); }
.sensor-card.water .card-value { color: var(--c-water); }
.sensor-card.rain .card-value { color: var(--c-rain); font-size: 22px; padding-top: 6px; }
.pill { display: inline-flex; align-items: center; gap: 5px; padding: 3px 10px; border-radius: 20px; font-size: 10px; font-weight: 600; font-family: var(--mono); text-transform: uppercase; letter-spacing: 0.06em; }
.pill.normal { background: rgba(54,201,142,.15); color: var(--green); }
.pill.warning { background: rgba(245,169,74,.15); color: var(--yellow); }
.pill.danger { background: rgba(240,98,82,.15); color: var(--red); }
.rain-yes { color: var(--purple); }
.rain-no { color: var(--muted); }
.section-title { font-size: 11px; text-transform: uppercase; letter-spacing: 0.1em; color: var(--muted); font-family: var(--mono); font-weight: 500; margin-bottom: 16px; display: flex; align-items: center; gap: 10px; }
.section-title::after { content: ''; flex: 1; height: 1px; background: var(--border-lo); }
.insights-grid { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 20px; }
@media (max-width: 960px) { .insights-grid { grid-template-columns: 1fr; } }
.insight-item { display: flex; align-items: flex-start; gap: 10px; padding: 10px 0; border-bottom: 1px solid var(--border-lo); }
.insight-item:last-child { border-bottom: none; }
.insight-icon { width: 30px; height: 30px; border-radius: 7px; display: grid; place-items: center; font-size: 14px; flex-shrink: 0; }
.insight-icon.info { background: rgba(91,156,246,.13); }
.insight-icon.warn { background: rgba(245,169,74,.13); }

```

```

.insight-icon.danger { background: rgba(240,98,82,.13); }
.insight-icon.ok { background: rgba(54,201,142,.13); }
.insight-text { font-size: 12px; line-height: 1.55; }
.insight-text strong { color: var(--text); font-weight: 500; }
.insight-text .time { color: var(--muted); font-size: 10px; font-family: var(--mono);
margin-top: 2px; display: block; }
.stat-row { display: flex; justify-content: space-between; align-items: center; padding: 7
border-bottom: 1px solid var(--border-lo); font-size: 12px; }
.stat-row:last-child { border-bottom: none; }
.stat-label { color: var(--muted); }
.stat-val { font-family: var(--mono); color: var(--text); font-size: 12px; }
.stat-val.up { color: var(--red); }
.stat-val.down { color: var(--green); }
.trend-arrow { display: inline-block; margin-left: 4px; font-size: 10px; }
.trend-up { color: var(--red); }
.trend-down { color: var(--green); }
.trend-flat { color: var(--muted); }
.event-log { max-height: 180px; overflow-y: auto; font-family: var(--mono); font-size: 11px
.event-log::-webkit-scrollbar { width: 4px; }
.event-log::-webkit-scrollbar-track { background: transparent; }
.event-log::-webkit-scrollbar-thumb { background: var(--border); border-radius: 2px; }
.event-entry { padding: 6px 0; border-bottom: 1px solid var(--border-lo); display: flex; gap: 10px; }
.event-entry:last-child { border-bottom: none; }
.event-time { color: var(--muted); white-space: nowrap; }
.event-msg { color: var(--text); }
.event-msg.warn { color: var(--yellow); }
.event-msg.danger { color: var(--red); }
.event-msg.ok { color: var(--green); }
.event-msg.info { color: var(--accent); }
.chart-wrap { position: relative; height: 200px; }
.chart-label { font-size: 11px; color: var(--muted); font-family: var(--mono); margin-bottom: 14px; letter-spacing: 0.05em; }
.toggle-row { display: flex; align-items: center; gap: 12px; margin-bottom: 14px; }
.toggle-row label:first-child { flex: 1; color: var(--muted); font-size: 12px; }
.toggle { position: relative; width: 42px; height: 24px; cursor: pointer; flex-shrink: 0; }
.toggle input { opacity: 0; width: 0; height: 0; }
.toggle-slider { position: absolute; inset: 0; background: var(--border); border-radius: 2
transition: background 0.25s; }
.toggle-slider::before { content: ''; position: absolute; left: 4px; top: 4px; width: 16px; height: 16px; background: #fff; border-radius: 50%; transition: transform 0.25s; box-shadow: 1px 3px 3px rgba(0,0,0,.3); }
.toggle input:checked + .toggle-slider { background: var(--accent); }
.toggle input:checked + .toggle-slider::before { transform: translateX(18px); }
.btn { display: inline-flex; align-items: center; gap: 6px; padding: 8px 16px; border-radius: var(--radius-sm); font-family: var(--mono); font-size: 12px; font-weight: 600; cursor: pointer; border: none; transition: opacity 0.15s, transform 0.1s, box-shadow 0.15s; letter-spacing: 0.02em; }
.btn:hover { opacity: .88; }
.btn:active { transform: scale(0.97); }
.btn-primary { background: var(--accent); color: #0e1520; box-shadow: 0 2px 10px rgba(91,156,246,.3); }
.btn-green { background: var(--green); color: #0e1520; box-shadow: 0 2px 10px rgba(54,201,142,.25); }
.btn-yellow { background: var(--yellow); color: #0e1520; }
.btn-red { background: var(--red); color: #fff; }
.btn-ghost { background: transparent; color: var(--muted); border: 1px solid var(--border); }
.btn-ghost:hover { color: var(--text); border-color: var(--muted); }
.btn-row { display: flex; gap: 10px; flex-wrap: wrap; margin-top: 6px; }
#toast { position: fixed; bottom: 24px; right: 24px; z-index: 100; background: var(--surface); border: 1px solid var(--border); color: var(--green); font-family: var(--mono); font-size: 12px; padding: 11px 20px; border-radius: 10px; opacity: 0; transform: translateY(8px); transition: opacity 0.3s, transform 0.3s; pointer-events: none; box-shadow: 0 8px 28px rgba(0,0,0,.4); }
#toast.show { opacity: 1; transform: translateY(0); }

```

```

#lastUpdated { font-family: var(--mono); font-size: 11px; color: var(--muted); }
.divider-col { border-left: 1px solid var(--border-lo); padding-left: 20px; }
@media (max-width: 960px) { .divider-col { border-left: none; padding-left: 0; } }
</style>
<base target="_blank">
</head>
<body>
<div class="shell">

  <header>
    <div class="logo">🌱</div>
    <div class="header-text">
      <h1>Smart Agriculture</h1>
      <p>IoT Monitoring Dashboard · esp32_01</p>
    </div>
    <div class="header-right">
      <div class="badge">
        <span class="dot" id="connDot"></span>
        <span id="connLabel">connecting...</span>
      </div>
      <button class="btn btn-ghost" style="font-size:11px;padding:6px 14px;"
onclick="doLogout()" >🚪 Logout</button>
    </div>
  </header>

  <div id="statusBar" class="normal">
    <span class="status-icon">●</span>
    <span id="statusMsg">Waiting for data...</span>
    <span style="margin-left:auto" id="lastUpdated">—</span>
  </div>

  <div class="grid-4">
    <div class="card sensor-card temp">
      <div class="sensor-icon">🌡️</div>
      <div class="card-label">Temperature</div>
      <div class="card-value" id="valTemp">—<span class="card-unit">°C</span></div>
      <div class="card-sub" id="subTemp">threshold: 31 °C <span id="trendTemp"></span></div>
    </div>
    <div class="card sensor-card hum">
      <div class="sensor-icon">💧</div>
      <div class="card-label">Humidity</div>
      <div class="card-value" id="valHum">—<span class="card-unit">%</span></div>
      <div class="card-sub" id="subHum">threshold: 70 % <span id="trendHum"></span></div>
    </div>
    <div class="card sensor-card water">
      <div class="sensor-icon">💧</div>
      <div class="card-label">Water Level</div>
      <div class="card-value" id="valWl">—<span class="card-unit">cm</span></div>
      <div class="card-sub" id="subWl">sensor range: 26-30 cm <span id="trendWl"></span></div>
    </div>
    <div class="card sensor-card rain">
      <div class="sensor-icon">🌧️</div>
      <div class="card-label">Rain / Drop</div>
      <div class="card-value" id="valRain">—</div>
      <div class="card-sub">GPIO 35 · DO signal</div>
    </div>
  </div>

  <div class="grid-1">
    <div class="card">
      <div class="section-title">Insights & Analysis</div>
      <div class="insights-grid">
        <div>
          <div class="card-label" style="margin-bottom:12px;">Smart Insights</div>
          <div id="insightsList">

```

```

        <div class="insight-item">
            <div class="insight-icon info">❗</div>
            <div class="insight-text"><strong>System initializing.</strong><span
class="time">Waiting for first sensor readings...</span></div>
        </div>
    </div>
    <div class="divider-col">
        <div class="card-label" style="margin-bottom:12px;">Session Statistics</div>
        <div class="stat-row"><span class="stat-label">Avg Temperature</span><span
class="stat-val" id="statAvgTemp">—</span></div>
        <div class="stat-row"><span class="stat-label">Avg Humidity</span><span
class="stat-val" id="statAvgHum">—</span></div>
        <div class="stat-row"><span class="stat-label">Max Water Level</span><span
class="stat-val" id="statMaxWl">—</span></div>
        <div class="stat-row"><span class="stat-label">Readings Received</span><span
class="stat-val" id="statCount">0</span></div>
        <div class="stat-row"><span class="stat-label">Danger Events</span><span
class="stat-val" id="statDanger" style="color:var(--red)">0</span></div>
        <div class="stat-row"><span class="stat-label">Warning Events</span><span
class="stat-val" id="statWarning" style="color:var(--yellow)">0</span></div>
    </div>
    <div class="divider-col">
        <div class="card-label" style="margin-bottom:12px;">Recent Events</div>
        <div class="event-log" id="eventLog">
            <div class="event-entry"><span class="event-time">--:--</span><span
class="event-msg">Dashboard loaded</span></div>
        </div>
    </div>
</div>
</div>

<div class="grid-2">
    <div class="card">
        <div class="chart-label">🌡️ Temperature & Humidity · last 30 readings</div>
        <div class="chart-wrap"><canvas id="chartTempHum"></canvas></div>
    </div>
    <div class="card">
        <div class="chart-label">🌊 Water Level · last 30 readings (sensor range 26-30 cm)</div>
        <div class="chart-wrap"><canvas id="chartWater"></canvas></div>
    </div>
</div>

<div class="grid-2">
    <div class="card">
        <div class="section-title">Manual Controls</div>
        <p style="color:var(--muted);font-size:12px;margin-bottom:18px;line-height:1.6;">
            Override servo states. The ESP32 syncs these within 2 seconds.
        </p>

        <div class="toggle-row">
            <label>Servo 1 — Container Lid Motor</label>
            <label class="toggle">
                <input type="checkbox" id="togServo1">
                <span class="toggle-slider"></span>
            </label>
        </div>
        <div class="toggle-row">
            <label>Servo 2 — Water Cap Motor</label>
            <label class="toggle">
                <input type="checkbox" id="togServo2">
                <span class="toggle-slider"></span>
            </label>
        </div>
    </div>

```

```

    </div>

    <div class="btn-row">
        <button type="button" class="btn btn-green" onclick="quickAction('open_container
event); return false;">🔓 Open Container</button>
        <button type="button" class="btn btn-ghost" onclick="quickAction('close_containe
event); return false;">🔒 Close Container</button>
    </div>
    <div class="btn-row" style="margin-top:10px;">
        <button type="button" class="btn btn-yellow" onclick="quickAction('open_drain',
event); return false;">💧 Open Drain</button>
        <button type="button" class="btn btn-ghost" onclick="quickAction('close_drain',
event); return false;">🚫 Close Drain</button>
    </div>
</div>

<div class="card">
    <div class="section-title">System Status</div>
    <p style="color:var(--muted);font-size:12px;margin-bottom:18px;line-height:1.6;">
        Current sensor thresholds are configured on the ESP32 firmware.
    </p>
    <div class="stat-row"><span class="stat-label">Temp Threshold</span><span
class="stat-val">31 °C</span></div>
    <div class="stat-row"><span class="stat-label">Humidity Threshold</span><span
class="stat-val">70 %</span></div>
    <div class="stat-row"><span class="stat-label">Water Alert</span><span
class="stat-val">26.5 cm</span></div>
    <div class="stat-row"><span class="stat-label">Upload Interval</span><span
class="stat-val">10 sec</span></div>
    <div class="stat-row"><span class="stat-label">Settings Sync</span><span
class="stat-val">2 sec</span></div>
</div>

</div>

<div id="toast">✓ Done</div>

<script>
/* — Config ————— */
const BASE = 'https://socstudentmusicforlife.com/fish_agriculture/php';

/* — Water Level Constants ————— */
const SENSOR_BOTTOM_CM = 26.0;
const SENSOR_RANGE_CM = 4.0;
const WATER_THRESHOLD_CM = 26.5;

/* — Heartbeat / Stale Data Detection ————— */
let lastDataTime = 0;
const HEARTBEAT_TIMEOUT = 8000;

/* — Water Level Conversion (ADC → cm) ————— */
function wLAdcToCm(adc) {
    if (adc == null || isNaN(adc)) return null;
    return SENSOR_BOTTOM_CM + (parseFloat(adc) / 4095.0) * SENSOR_RANGE_CM;
}

/* — Chart setup ————— */
const chartDefaults = {
    responsive: true, maintainAspectRatio: false,
    animation: { duration: 300 },
    plugins: { legend: { labels: { color: '#7a8fb8', font: { family: 'JetBrains Mono', size:
} } } },
    scales: {

```

```

        x: { ticks: { color: '#7a8fb8', font: { family: 'JetBrains Mono', size: 9 } }, grid: {
color: '#2e3f6a44' } },
        y: { ticks: { color: '#7a8fb8', font: { family: 'JetBrains Mono', size: 9 } }, grid: {
color: '#2e3f6a44' } },
    }
};

const thCtx = document.getElementById('chartTempHum').getContext('2d');
const wlCtx = document.getElementById('chartWater').getContext('2d');

const chartTH = new Chart(thCtx, {
    type: 'line',
    data: {
        labels: [],
        datasets: [
            { label: 'Temp °C', data: [], borderColor: '#f06252', backgroundColor:
'rgba(240,98,82,.08)', tension: 0.35, pointRadius: 2 },
            { label: 'Humidity %', data: [], borderColor: '#5b9cf6', backgroundColor:
'rgba(91,156,246,.08)', tension: 0.35, pointRadius: 2 }
        ]
    },
    options: { ...chartDefaults }
});

const chartWL = new Chart(wlCtx, {
    type: 'line',
    data: {
        labels: [],
        datasets: [
            { label: 'Water (cm)', data: [], borderColor: '#36c98e', backgroundColor:
'rgba(54,201,142,.08)', tension: 0.35, pointRadius: 2 }
        ]
    },
    options: {
        ...chartDefaults,
        scales: {
            x: { ticks: { color: '#7a8fb8', font: { family: 'JetBrains Mono', size: 9 } }, grid:
color: '#2e3f6a44' } },
            y: {
                min: 26,
                max: 30,
                ticks: { color: '#7a8fb8', font: { family: 'JetBrains Mono', size: 9 }, stepSize: 1
},
                grid: { color: '#2e3f6a44' },
                title: { display: true, text: 'Distance from bottom (cm)', color: '#7a8fb8', font:
family: 'JetBrains Mono', size: 10 } }
        }
    }
});

const MAX_POINTS = 30;

/* — Analytics State ————— */
const history = [];
let dangerCount = 0;
let warningCount = 0;
let lastStatus = 'Normal';

/* — SSE Live Feed ————— */
let sse;
function startSSE() {
    if (sse) sse.close();
    sse = new EventSource(BASE + '/api_stream.php', { withCredentials: true });

    sse.onopen = () => {};

```

```

sse.onerror = () => { setConnected(false); setTimeout(startSSE, 5000); };

sse.onmessage = (e) => {
    const d = JSON.parse(e.data);
    lastDataTime = Date.now();
    setConnected(true);
    updateCards(d);
    pushToCharts(d);
    analyze(d);
};
}

setInterval(() => {
    if (Date.now() - lastDataTime > HEARTBEAT_TIMEOUT) {
        setConnected(false);
    }
}, 2000);

/* — Load history for charts on startup ————— */
async function loadHistory() {
    try {
        const r = await fetch(BASE + '/api_get_history.php?limit=30', { credentials: 'include' });
        const rows = await r.json();
        rows.forEach(d => {
            pushToCharts(d);
            history.push(d);
        });
        if (history.length) {
            lastDataTime = Date.now();
            analyze(history[history.length - 1]);
        }
    } catch {}
}

/* — Update UI ————— */
function updateCards(d) {
    setText('valTemp', fmt(d.temperature, 1) + '<span class="card-unit">°C</span>');
    setText('valHum', fmt(d.humidity, 1) + '<span class="card-unit">%</span>');

    const wlCm = wlAdcToCm(d.water_level);
    setText('valWl', wlCm != null ? wlCm.toFixed(1) + '<span class="card-unit">cm</span>' : ' ');

    setText('valRain', d.water_drop == 1
        ? '<span class="rain-yes">💧 Yes</span>'
        : '<span class="rain-no">🚰 No</span>');

    const dateTime = d.created_at ? new Date(d.created_at).getTime() : Date.now();
    const ageSec = (Date.now() - dateTime) / 1000;
    const ageMin = ageSec / 60;

    const s = (d.status || 'Normal').toLowerCase();
    const bar = document.getElementById('statusBar');

    if (ageSec > 120) {
        bar.className = 'stale';
        document.getElementById('statusMsg').textContent =
            '🕒 Last data ' + (ageMin < 1 ? Math.round(ageSec) + ' sec' : Math.round(ageMin) + ' r'
+ ' ago — ESP32 offline');
    } else {
        bar.className = s;
        document.getElementById('statusMsg').textContent =
            s === 'normal' ? '✅ Room environment stable' :
            s === 'warning' ? '⚠️ Warning — check readings' : '🚫 Danger — immediate action needed';
    }
}

```

```

document.getElementById('lastUpdated').textContent =
    'Updated ' + new Date().toLocaleTimeString();
}

function pushToCharts(d) {
    const ts = d.created_at ? d.created_at.slice(11,16) : new
Date().toLocaleTimeString().slice(0,5);

    [chartTH, chartWL].forEach(c => {
        c.data.labels.push(ts);
        if (c.data.labels.length > MAX_POINTS) c.data.labels.shift();
    });

    chartTH.data.datasets[0].data.push(d.temperature);
    chartTH.data.datasets[1].data.push(d.humidity);
    chartTH.data.datasets.forEach(ds => { if (ds.data.length > MAX_POINTS) ds.data.shift(); });

    chartWL.data.datasets[0].data.push(wlAdcToCm(d.water_level));
    chartWL.data.datasets.forEach(ds => { if (ds.data.length > MAX_POINTS) ds.data.shift(); });

    chartTH.update('none');
    chartWL.update('none');
}

function setConnected(ok) {
    document.getElementById('connDot').className = 'dot' + (ok ? '' : ' offline');
    document.getElementById('connLabel').textContent = ok ? 'live' : 'offline';
}

/* — Analysis Engine ————— */
function analyze(d) {
    history.push(d);
    if (history.length > 60) history.shift();

    const temps = history.filter(x => x.temperature !== null).map(x => x.temperature);
    const hums   = history.filter(x => x.humidity !== null).map(x => x.humidity);
    const wls    = history.filter(x => x.water_level !== null).map(x => wlAdcToCm(x.water_level));

    const avgT   = temps.length ? (temps.reduce((a,b)=>a+b,0)/temps.length).toFixed(1) : '-';
    const avgH   = hums.length ? (hums.reduce((a,b)=>a+b,0)/hums.length).toFixed(1) : '-';
    const maxWl  = wls.length ? Math.max(...wls).toFixed(1) : '-';

    document.getElementById('statAvgTemp').textContent = avgT + (temps.length ? ' °C' : '');
    document.getElementById('statAvgHum').textContent = avgH + (hums.length ? ' %' : '');
    document.getElementById('statMaxWl').textContent = maxWl + (wls.length ? ' cm' : '');
    document.getElementById('statCount').textContent = history.length;
    document.getElementById('statDanger').textContent = dangerCount;
    document.getElementById('statWarning').textContent = warningCount;

    const trend = (arr, elId) => {
        if (arr.length < 6) { document.getElementById(elId).innerHTML = ''; return; }
        const recent = arr.slice(-5).reduce((a,b)=>a+b,0)/5;
        const prev   = arr.slice(-10,-5).reduce((a,b)=>a+b,0)/5;
        const diff = recent - prev;
        const arrow = Math.abs(diff) < 0.3 ? '<span class="trend-arrow trend-flat">↔</span>' :
            diff > 0 ? '<span class="trend-arrow trend-up">↗ + ' + diff.toFixed(1) +
'</span>' :
            '<span class="trend-arrow trend-down">↘ ' + diff.toFixed(1) + '</span>';
        document.getElementById(elId).innerHTML = arrow;
    };
    trend(temps, 'trendTemp');
    trend(hums, 'trendHum');
    trend(wls, 'trendWl');

    const status = (d.status || 'Normal').toLowerCase();

```



```

    if (status === 'danger' && lastStatus !== 'danger') {
        dangerCount++;
        const wlCm = wlAdcToCm(d.water_level);
        logEvent('⚠ Danger: water level threshold exceeded (' + (wlCm !== null ? wlCm.toFixed(1)
cm' : d.water_level) + '), 'danger');
    } else if (status === 'warning' && lastStatus !== 'warning') {
        warningCount++;
        const reason = d.water_drop == 1 ? 'rain detected' : 'temperature high';
        logEvent('⚠ Warning: ' + reason, 'warn');
    } else if (status === 'normal' && lastStatus !== 'normal') {
        logEvent('✓ System returned to normal', 'ok');
    }
    lastStatus = status;

    const insights = [];
    const now = new Date();
    const timeStr = now.toLocaleTimeString().slice(0,5);

    const wlCm = wlAdcToCm(d.water_level);

    const dataTime = d.created_at ? new Date(d.created_at).getTime() : Date.now();
    const ageMin = (Date.now() - dataTime) / 60000;

    if (ageMin > 2) {
        insights.push({ icon: '🕒', cls: 'warn', text: '<strong>Stale data.</strong> Last reading
' + Math.round(ageMin) + ' minutes ago. ESP32 may be powered off or disconnected.', time:
timeStr });
    }

    if (wlCm !== null && wlCm > WATER_THRESHOLD_CM) {
        insights.push({ icon: '⚠', cls: 'danger', text: '<strong>Water level alert.</strong> '
wlCm.toFixed(1) + ' cm) Drain cap motor has been triggered.', time: timeStr });
    }
    if (d.water_drop == 1) {
        insights.push({ icon: '💧', cls: 'warn', text: '<strong>Rain detected.</strong> Containr
lid motor opened for protection.', time: timeStr });
    }
    if (d.temperature > 31) {
        insights.push({ icon: '🔥', cls: 'warn', text: '<strong>High room temperature.</strong>
opened for cooling ventilation.', time: timeStr });
    }
    if (d.humidity > 70) {
        insights.push({ icon: '💧', cls: 'info', text: '<strong>High room humidity.</strong> Mo
for condensation.', time: timeStr });
    }

    if (temps.length >= 10) {
        const last3 = temps.slice(-3).reduce((a,b)=>a+b,0)/3;
        const prev3 = temps.slice(-6,-3).reduce((a,b)=>a+b,0)/3;
        if (last3 > prev3 + 2) {
            insights.push({ icon: '📈', cls: 'info', text: '<strong>Temperature rising fast.</str
+ ' + (last3-prev3).toFixed(1) + ' °C over last 3 readings.', time: timeStr });
        }
    }
    if (wls.length >= 10) {
        const last3 = wls.slice(-3).reduce((a,b)=>a+b,0)/3;
        const prev3 = wls.slice(-6,-3).reduce((a,b)=>a+b,0)/3;
        if (Math.abs(last3 - prev3) > 0.5) {
            const dir = last3 > prev3 ? 'rising' : 'falling';
            insights.push({ icon: '📉', cls: 'warn', text: '<strong>Water level ' + dir + '.</str
' + Math.abs(last3-prev3).toFixed(1) + ' cm change.', time: timeStr });
        }
    }

    if (insights.length === 0) {

```

```

    insights.push({ icon: '✓', cls: 'ok', text: '<strong>All systems nominal.</strong> No
alerts. Container environment is stable.', time: timeStr });
}

const list = document.getElementById('insightsList');
list.innerHTML = insights.map(i => `
    <div class="insight-item">
        <div class="insight-icon ${i.cls}">${i.icon}</div>
        <div class="insight-text">${i.text}<span class="time">${i.time}</span></div>
    </div>
`).join('');
}

function logEvent(msg, cls) {
    const log = document.getElementById('eventLog');
    const time = new Date().toLocaleTimeString().slice(0,5);
    const entry = document.createElement('div');
    entry.className = 'event-entry';
    entry.innerHTML = `<span class="event-time">${time}</span><span class="event-msg
${cls}">${msg}</span>`;
    log.insertBefore(entry, log.firstChild);
    while (log.children.length > 20) log.removeChild(log.lastChild);
}

/* — Manual Controls ————— */
async function quickAction(action, event) {
    // Hard-block any default browser action (form submit, navigation, etc.)
    if (event) {
        event.preventDefault();
        event.stopPropagation();
    }

    const map = {
        open_container: { servo1_open: 1, servo2_open: 0 },
        close_container: { servo1_open: 0, servo2_open: 0 },
        open_drain: { servo1_open: 0, servo2_open: 1 },
        close_drain: { servo1_open: 0, servo2_open: 0 },
    };
    const states = map[action];
    if (!states) return;

    // Sync toggles
    document.getElementById('togServo1').checked = !!states.servo1_open;
    document.getElementById('togServo2').checked = !!states.servo2_open;

    const payload = {
        device_id: 'esp32_01',
        temp_threshold: 31.0,
        humidity_threshold: 70.0,
        water_level_threshold: 26.5,
        upload_interval: 10,
        buzzer_enabled: 1,
        servo1_open: states.servo1_open,
        servo2_open: states.servo2_open,
    };

    try {
        const res = await fetch(BASE + '/api_update_settings.php', {
            credentials: 'include',
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
    }

    // If the server returns an error status, read the body and throw

```

```

    if (!res.ok) {
        const text = await res.text();
        let msg = 'HTTP ' + res.status;
        try { const j = JSON.parse(text); if (j.error) msg = j.error; } catch {}
        throw new Error(msg);
    }

    toast('Command sent - ' + action.replace(/_/g, ' '));
    logEvent('🔧 Motor command: ' + action.replace(/_/g, ' '), 'info');
} catch (err) {
    console.error('quickAction failed:', err);
    toast('Failed: ' + err.message);
}

return false; // extra safety for inline handler
}

async function sendServoState() {
    const payload = {
        device_id: 'esp32_01',
        temp_threshold: 31.0,
        humidity_threshold: 70.0,
        water_level_threshold: 26.5,
        upload_interval: 10,
        buzzer_enabled: 1,
        servo1_open: document.getElementById('togServo1').checked ? 1 : 0,
        servo2_open: document.getElementById('togServo2').checked ? 1 : 0,
    };
    try {
        const res = await fetch(BASE + '/api_update_settings.php', {
            credentials: 'include',
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        if (!res.ok) throw new Error('HTTP ' + res.status);
        toast('Settings updated');
    } catch (err) {
        console.error('sendServoState failed:', err);
        toast('Update failed');
    }
}

document.getElementById('togServo1').addEventListener('change', sendServoState);
document.getElementById('togServo2').addEventListener('change', sendServoState);

/* — Helpers — */
function fmt(v, d) { return (v == null || isNaN(v)) ? '-' : parseFloat(v).toFixed(d); }
function setText(id, html) { document.getElementById(id).innerHTML = html; }

function toast(msg) {
    const el = document.getElementById('toast');
    el.textContent = '✓ ' + msg;
    el.classList.add('show');
    setTimeout(() => el.classList.remove('show'), 2800);
}

/* — Auth Guard — */
async function checkAuth() {
    try {
        const r = await fetch(BASE + '/api_check_auth.php', { credentials: 'include' });
        if (!r.ok) throw new Error('Unauthorized');
        const data = await r.json();
        if (!data.authenticated) throw new Error('Unauthorized');
    } catch (e) {

```

```

        window.location.href = 'login.html';
    }
}

async function doLogout() {
    await fetch(BASE + '/api_logout.php', { method: 'POST', credentials: 'include' });
    window.location.href = 'login.html';
}

/* — Boot ————— */
loadHistory();
startSSE();
checkAuth();
</script>
</body>
</html>

```

Login.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>SmartContainer · Login</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link
href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500;600&family=Inter@300;400;500;600;700&display=swap" rel="stylesheet">
<style>
:root {
    --bg: #151d2e;
    --surface: #1c2540;
    --surface2: #243058;
    --border: #2e3f6a;
    --border-lo:#253250;
    --accent: #5b9cf6;
    --green: #36c98e;
    --red: #f06252;
    --text: #dde5f8;
    --muted: #7a8fb8;
    --mono: 'JetBrains Mono', monospace;
    --sans: 'Inter', sans-serif;
}

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

body {
    background: var(--bg);
    background-image:
        radial-gradient(ellipse 70% 50% at 50% -10%, rgba(91,156,246,.12) 0%, transparent 65%),
        radial-gradient(ellipse 40% 40% at 10% 90%, rgba(54,201,142,.06) 0%, transparent 60%);
    color: var(--text);
    font-family: var(--sans);
    font-size: 14px;
    min-height: 100vh;
    display: grid;
    place-items: center;
}

```

```

/* — Decorative sensor strip (top) ————— */
.sensor-strip {
  position: fixed;
  top: 0; left: 0; right: 0;
  height: 3px;
  background: linear-gradient(90deg,
    #f06252 0%, #f06252 25%,
    #5b9cf6 25%, #5b9cf6 50%,
    #36c98e 50%, #36c98e 75%,
    #a78bfa 75%, #a78bfa 100%
  );
  opacity: .7;
}

/* — Card ————— */
.login-wrap {
  width: 100%;
  max-width: 400px;
  padding: 20px;
}

.login-card {
  background: var(--surface);
  border: 1px solid var(--border-lo);
  border-radius: 16px;
  padding: 42px 38px 36px;
  box-shadow: 0 20px 60px rgba(0,0,0,.45), 0 0 0 1px rgba(91,156,246,.04);
}

/* — Logo ————— */
.logo-wrap {
  display: flex;
  align-items: center;
  gap: 14px;
  margin-bottom: 32px;
}

.logo {
  width: 44px; height: 44px;
  background: linear-gradient(135deg, var(--accent) 0%, #2563eb 100%);
  border-radius: 11px;
  display: grid; place-items: center;
  font-size: 22px;
  box-shadow: 0 6px 18px rgba(91,156,246,.3);
  flex-shrink: 0;
}

.logo-text h1 {
  font-size: 18px;
  font-weight: 700;
  letter-spacing: -0.4px;
  color: var(--text);
}

.logo-text p {
  color: var(--muted);
  font-size: 11px;
  font-family: var(--mono);
  margin-top: 2px;
}

/* — Sensor indicators ————— */
.sensor-dots {
  display: flex;
  gap: 6px;
  margin-bottom: 28px;
}

.sensor-dot {

```

```

display: flex;
align-items: center;
gap: 5px;
padding: 4px 10px;
border-radius: 20px;
font-size: 10px;
font-family: var(--mono);
font-weight: 500;
letter-spacing: 0.04em;
}
.sensor-dot .pip {
width: 5px; height: 5px;
border-radius: 50%;
animation: pip-pulse 2.4s ease infinite;
}
.sensor-dot.temp { background: rgba(240,98,82,.1); color: #f06252; }
.sensor-dot.temp .pip { background: #f06252; animation-delay: 0s; }
.sensor-dot.hum { background: rgba(91,156,246,.1); color: #5b9cf6; }
.sensor-dot.hum .pip { background: #5b9cf6; animation-delay: .4s; }
.sensor-dot.water { background: rgba(54,201,142,.1); color: #36c98e; }
.sensor-dot.water .pip { background: #36c98e; animation-delay: .8s; }

@keyframes pip-pulse {
0%,100% { opacity: 1; }
50% { opacity: .25; }
}

/* — Fields ————— */
.field { margin-bottom: 18px; }
.field label {
display: block;
color: var(--muted);
font-size: 10px;
text-transform: uppercase;
letter-spacing: 0.1em;
font-family: var(--mono);
font-weight: 500;
margin-bottom: 7px;
}
.field input {
width: 100%;
background: var(--bg);
border: 1px solid var(--border);
color: var(--text);
font-family: var(--mono);
font-size: 14px;
padding: 11px 14px;
border-radius: 9px;
outline: none;
transition: border-color .2s, box-shadow .2s;
}
.field input::placeholder { color: var(--border); }
.field input:focus {
border-color: var(--accent);
box-shadow: 0 0 0 3px rgba(91,156,246,.1);
}

/* — Button ————— */
.btn {
width: 100%;
padding: 12px;
border-radius: 9px;
border: none;
background: var(--accent);
color: #0e1520;

```

```

    font-family: var(--mono);
    font-size: 13px;
    font-weight: 600;
    cursor: pointer;
    transition: opacity .15s, transform .1s, box-shadow .15s;
    box-shadow: 0 4px 16px rgba(91,156,246,.35);
    letter-spacing: 0.03em;
    margin-top: 6px;
}
.btn:hover { opacity: .88; box-shadow: 0 6px 20px rgba(91,156,246,.4); }
.btn:active { transform: scale(0.98); }
.btn:disabled { opacity: .45; cursor: not-allowed; box-shadow: none; }

/* --- Error ----- */
.error {
    margin-top: 14px;
    padding: 10px 14px;
    border-radius: 8px;
    background: rgba(240,98,82,.08);
    border: 1px solid rgba(240,98,82,.2);
    color: var(--red);
    font-family: var(--mono);
    font-size: 12px;
    display: none;
}
.error.show { display: block; }

/* --- Hint ----- */
.hint {
    margin-top: 20px;
    text-align: center;
    color: var(--border);
    font-size: 10px;
    font-family: var(--mono);
    letter-spacing: 0.04em;
}
.hint span { color: var(--muted); }
</style>
</head>
<body>

<div class="sensor-strip"></div>

<div class="login-wrap">
<div class="login-card">

    <div class="logo-wrap">
        <div class="logo">🌱</div>
        <div class="logo-text">
            <h1>SmartAgriculture</h1>
            <p>IoT Monitoring Dashboard</p>
        </div>
    </div>

    <div class="field">
        <label>Username</label>
        <input type="text" id="username" placeholder="admin" autocomplete="username">
    </div>
    <div class="field">
        <label>Password</label>
        <input type="password" id="password" placeholder="....."
autocomplete="current-password">
    </div>

    <button class="btn" id="loginBtn" onclick="doLogin()">Sign In -</button>

```

```

    <div class="error" id="errorBox">Invalid credentials</div>

    <div class="hint">Default: <span>admin</span> / <span>password</span></div>
  </div>
</div>

<script>
const BASE = 'https://socstudentmusicforlife.com/fish_agriculture/php'; // ← CHANGE folde
needed

async function doLogin() {
  const btn = document.getElementById('loginBtn');
  const err = document.getElementById('errorBox');
  const u = document.getElementById('username').value.trim();
  const p = document.getElementById('password').value;

  btn.disabled = true;
  err.classList.remove('show');

  try {
    const r = await fetch(BASE + '/api_login.php', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      credentials: 'include',
      body: JSON.stringify({ username: u, password: p })
    });
    const data = await r.json();
    if (data.success) {
      window.location.href = 'index.html';
    } else {
      err.textContent = data.error || 'Invalid credentials';
      err.classList.add('show');
    }
  } catch (e) {
    err.textContent = 'Network error';
    err.classList.add('show');
  } finally {
    btn.disabled = false;
  }
}

document.getElementById('password').addEventListener('keydown', e => {
  if (e.key === 'Enter') doLogin();
});
</script>
</body>
</html>

```

SmartAgriculture.ino


```

/**
 *
 *   IoT Smart Agriculture - ESP32 Firmware v2.4
 *   DHT22 · Water Level · Rain · Touch · 2× Servo · Buzzer · OLED
 *   WiFiManager portal + Preferences + Min/Max + Fast reconnect
 *
 */

#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "DHT.h"

// ===== WiFiManager + Preferences =====
#include <WiFiManager.h>
#include <Preferences.h>

Preferences prefs;
bool shouldSaveConfig = false;
WiFiManager wm;
// =====

// Server
String serverHost = "https://socstudentmusicforlife.com/fish_agriculture";
String deviceID   = "esp32_01";

// GPIO Pin Map
#define PIN_SERVO_LID           4
#define PIN_SERVO_DRAIN        16
#define PIN_WATER_LEVEL_POWER  14
#define PIN_WATER_LEVEL_SIGNAL 32
#define PIN_RAIN_POWER          33
#define PIN_RAIN_ANALOG         34
#define PIN_DHT                 19
#define PIN_OLED_SDA            21
#define PIN_OLED_SCL            22
#define PIN_BUZZER              5
#define PIN_TOUCH               17

#define DHT_TYPE DHT22

// Water Level Calibration
#define SENSOR_RANGE_CM         4
#define SENSOR_TOP_CM           28
#define SENSOR_BOTTOM_CM        26.0
#define WATER_THRESHOLD_CM      26.5

// Rain Sensor Constants
#define RAIN_WET_THRESHOLD      1500
#define RAIN_DRY_THRESHOLD      3500
#define RAIN_INVALID_MAX        100
#define RAIN_DEBOUNCE_MS        500

// Servo and Buzzer Configuration
#define SERVO_FREQUENCY         50
#define SERVO_RESOLUTION        16
#define BUZZER_FREQUENCY        2000
#define BUZZER_RESOLUTION       8

// Servo Positions
#define LID_CLOSED_ANGLE        0
#define LID_OPEN_ANGLE          180
#define DRAIN_CLOSED_ANGLE      0

```

```

#define DRAIN_OPEN_ANGLE    180

// Touch Debounce
#define TOUCH_COOLDOWN_MS  3000

// Objects
DHT dhtSensor(PIN_DHT, DHT_TYPE);
Adafruit_SSD1306 oledScreen(128, 64, &Wire, -1);

// Timing
unsigned long previousMillis      = 0;
unsigned long lastSettingsFetch   = 0;
unsigned long lastSensorReadTime  = 0;
unsigned long oledMessageTimeout  = 0;
unsigned long lastTouchToggleTime = 0;
int          uploadInterval       = 10;

// Sensor Values
float temperature = NAN;
float humidity    = NAN;
int   waterLevel  = 0;
float waterHeightCm = 0.0;
int   rainAnalogValue = 4095;
bool  isRaining    = false;
bool  isTouched     = false;

// Min/Max Tracking
float minTemp = NAN, maxTemp = NAN;
float minHum  = NAN, maxHum  = NAN;
bool  statsReady = false;

// Settings (synced from server)
float tempThreshold    = 31.0;
float humidityThreshold = 70.0;
int   waterThreshold   = 500;
bool  buzzerEnabled    = true;
bool  drainOpenOverride = false;
bool  lidOpenOverride  = false;

// Manual / State Flags
bool  lidManualOpen    = false;
bool  lastRainState    = false;
bool  rainBuzzerFired  = false;
bool  isOnlineMode     = false;

// Servo Tracking
int  lidServoPosition  = LID_CLOSED_ANGLE;
int  drainServoPosition = DRAIN_CLOSED_ANGLE;

// Non-blocking Buzzer Queue
struct BuzzPattern {
    uint32_t toneFrequency;
    uint16_t beepDuration;
    uint16_t pauseDuration;
    uint8_t  repeatCount;
};
BuzzPattern buzzQueue[4];
uint8_t buzzQueueHead      = 0;
uint8_t buzzQueueCount     = 0;
uint8_t buzzRepeatCounter  = 0;
bool    buzzerIsOn         = false;
unsigned long buzzerNextAction = 0;

// -----
// Servo Helpers

```

```

// =====
uint32_t angleToDuty(int angle) {
    uint32_t pulseMicros = 1000 + ((uint32_t)(2000 - 1000) * angle / 180);
    return (uint32_t)(pulseMicros * 65535UL / 20000UL);
}

void openLid() { lidServoPosition = LID_OPEN_ANGLE; ledcWrite(PIN_SERVO_LID,
angleToDuty(lidServoPosition)); }
void closeLid() { lidServoPosition = LID_CLOSED_ANGLE; ledcWrite(PIN_SERVO_LID,
angleToDuty(lidServoPosition)); }
void openDrain() { drainServoPosition = DRAIN_OPEN_ANGLE; ledcWrite(PIN_SERVO_DRAIN,
angleToDuty(drainServoPosition)); }
void closeDrain() { drainServoPosition = DRAIN_CLOSED_ANGLE; ledcWrite(PIN_SERVO_DRAIN,
angleToDuty(drainServoPosition)); }
bool isLidOpen() { return lidServoPosition == LID_OPEN_ANGLE; }
bool isDrainOpen() { return drainServoPosition == DRAIN_OPEN_ANGLE; }

// =====
// Buzzer Helpers
// =====
void buzzerTone(uint32_t freq) {
    if (freq == 0) ledcWrite(PIN_BUZZER, 0);
    else ledcWriteTone(PIN_BUZZER, freq);
}

void queueBuzzer(uint32_t freq, uint16_t onMs, uint16_t offMs, uint8_t repeats) {
    if (!buzzerEnabled || buzzQueueCount >= 4) return;
    buzzQueue[(buzzQueueHead + buzzQueueCount) % 4] = { freq, onMs, offMs, repeats };
    buzzQueueCount++;
}

void updateBuzzer() {
    if (buzzQueueCount == 0) return;
    if (millis() < buzzerNextAction) return;
    BuzzPattern& current = buzzQueue[buzzQueueHead];
    if (!buzzerIsOn) {
        buzzerTone(current.toneFrequency);
        buzzerIsOn = true;
        buzzerNextAction = millis() + current.beepDuration;
    } else {
        buzzerTone(0);
        buzzerIsOn = false;
        buzzRepeatCounter++;
        if (buzzRepeatCounter >= current.repeatCount) {
            buzzRepeatCounter = 0;
            buzzQueueHead = (buzzQueueHead + 1) % 4;
            buzzQueueCount--;
        } else {
            buzzerNextAction = millis() + current.pauseDuration;
        }
    }
}

// =====
// OLED Helpers
// =====
void showStartupScreen() {
    oledScreen.clearDisplay();
    oledScreen.setTextColor(WHITE);
    oledScreen.setTextSize(1);
    oledScreen.setCursor(14, 10);
    oledScreen.println("SmartAgriculture");
    oledScreen.setCursor(32, 26);
    oledScreen.println("Starting...");
    oledScreen.display();
}

void showTemporaryMessage(String message, unsigned long durationMs) {
    oledMessageTimeout = millis() + durationMs;
}

```

```

oledScreen.clearDisplay();
oledScreen.setTextColor(WHITE);
oledScreen.setTextSize(1);
oledScreen.setCursor(0, 0);
oledScreen.println(message);
oledScreen.display();
}

void refreshOledScreen() {
    if (millis() < oledMessageTimeout) return;
    oledScreen.clearDisplay();
    oledScreen.setTextColor(WHITE);
    oledScreen.setTextSize(1);
    oledScreen.setCursor(0, 0); oledScreen.printf("RoomT:%.1fC", temperature);
    oledScreen.setCursor(0, 12); oledScreen.printf("RoomH:%.1f%", humidity);
    oledScreen.setCursor(0, 24); oledScreen.printf("Water:%.1fcm", waterHeightCm);
    oledScreen.setCursor(0, 36); oledScreen.printf("Rain:%s", isRaining ? "YES" : "r");
    oledScreen.setCursor(0, 48); oledScreen.printf("Lid:%s Drain:%s",
        isLidOpen() ? "OPEN" : "CLSD",
        isDrainOpen() ? "OPEN" : "CLSD");
    oledScreen.setCursor(0, 56);
    oledScreen.print(isOnlineMode ? "Mode:ONLINE" : "Mode:OFFLINE");
    oledScreen.display();
}

// =====
// WiFiManager Callback
// =====
void saveConfigCallback() {
    Serial.println("WiFiManager: Config should be saved");
    shouldSaveConfig = true;
}

// =====
// WiFi (WiFiManager + Preferences)
// =====
void connectWiFi() {
    // 1. Try saved credentials from Preferences
    prefs.begin("smartagriculture", false);
    String savedSSID = prefs.getString("ssid", "");
    String savedPass = prefs.getString("pass", "");
    prefs.end();

    if (savedSSID.length() > 0) {
        Serial.print("Loaded saved WiFi: "); Serial.println(savedSSID);
        WiFi.mode(WIFI_STA);
        WiFi.begin(savedSSID.c_str(), savedPass.c_str());
        int retry = 0;
        // =====
        // delay(250) for faster boot
        // =====
        while (WiFi.status() != WL_CONNECTED && retry < 30) {
            delay(250);
            Serial.print(".");
            retry++;
        }
        if (WiFi.status() == WL_CONNECTED) {
            isOnlineMode = true;
            Serial.println("\nWiFi connected (saved creds).");
            Serial.print("IP: "); Serial.println(WiFi.localIP());
            showTemporaryMessage("ONLINE\n" + WiFi.localIP().toString(), 3000);
            return;
        }
        Serial.println("\nSaved WiFi failed. Starting portal...");
    }
}

```

```

// 2. Start captive portal for configuration
Serial.println("Starting WiFiManager captive portal...");
showTemporaryMessage("AP Mode\nConnect to setup", 3000);

wm.setConfigPortalTimeout(180);
wm.setAPCallback([](WiFiManager* myWiFiManager) {
    Serial.print("AP IP: "); Serial.println(WiFi.softAPIP());
    showTemporaryMessage("AP: SmartAgriculture\nIP: 192.168.4.1", 5000);
});
wm.setSaveConfigCallback(saveConfigCallback);

WiFiManagerParameter custom_device_id("device_id", "Device ID", deviceID.c_str(), 20);
wm.addParameter(&custom_device_id);

bool res = wm.autoConnect("SmartAgriculture-Setup");

if (!res) {
    Serial.println("Failed to connect. Rebooting...");
    showTemporaryMessage("Setup failed\nRebooting...", 3000);
    delay(3000);
    ESP.restart();
}

// 3. Connected via portal
isOnlineMode = true;
Serial.println("WiFi connected via portal.");
Serial.print("IP: "); Serial.println(WiFi.localIP());

// 4. Save new credentials to Preferences
if (shouldSaveConfig) {
    prefs.begin("smartagriculture", false);
    prefs.putString("ssid", WiFi.SSID());
    prefs.putString("pass", WiFi.psk());
    prefs.putString("device_id", custom_device_id.getValue());
    prefs.end();
    Serial.println("Config saved to flash.");
    deviceID = String(custom_device_id.getValue());
    shouldSaveConfig = false;
}

showTemporaryMessage("ONLINE\n" + WiFi.localIP().toString(), 3000);
}

// -----
// setup()
// -----
void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("\nESP32 Smart Agriculture v2.4");
    Serial.println("-----");

    ledcAttach(PIN_BUZZER, BUZZER_FREQUENCY, BUZZER_RESOLUTION);
    buzzerTone(0);

    ledcAttach(PIN_SERVO_LID, SERVO_FREQUENCY, SERVO_RESOLUTION);
    ledcAttach(PIN_SERVO_DRAIN, SERVO_FREQUENCY, SERVO_RESOLUTION);
    closeLid();
    closeDrain();

    pinMode(PIN_WATER_LEVEL_POWER, OUTPUT); digitalWrite(PIN_WATER_LEVEL_POWER, LOW);
    pinMode(PIN_RAIN_POWER, OUTPUT); digitalWrite(PIN_RAIN_POWER, LOW);
    pinMode(PIN_TOUCH, INPUT);
    dhtSensor.begin();
}

```

```

Wire.begin(PIN_OLED_SDA, PIN_OLED_SCL);
if (!oledScreen.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED init failed");
    while (true) delay(100);
}
showStartupScreen();
connectWiFi();
}

// =====
// || loop() - FIXED: Non-blocking WiFi reconnect for fast response ||
// =====

void loop() {
    unsigned long now = millis();

    // =====
    // WiFi Management: NON-BLOCKING reconnect
    // =====

    static unsigned long lastReconnectAttempt = 0;
    static bool wasOffline = false;

    if (WiFi.status() != WL_CONNECTED) {
        // Try to reconnect every 10 seconds - but DON'T block
        if (now - lastReconnectAttempt > 10000) {
            lastReconnectAttempt = now;

            if (isOnlineMode) {
                Serial.println("WiFi LOST. Switching to offline...");
                isOnlineMode = false;
                wasOffline = true;
                showTemporaryMessage("OFFLINE\nWiFi lost", 3000);
            } else if (wasOffline) {
                Serial.println("Still offline. Retrying...");
            }

            // Start reconnect (non-blocking - returns immediately)
            if (WiFi.getMode() != WIFI_STA) {
                WiFi.mode(WIFI_STA);
            }
            WiFi.reconnect();
        }

        // Check if reconnected (without blocking)
        if (WiFi.status() == WL_CONNECTED) {
            isOnlineMode = true;
            wasOffline = false;
            Serial.println("WiFi RECONNECTED!");
            Serial.print("IP: "); Serial.println(WiFi.localIP());
            showTemporaryMessage("ONLINE\n" + WiFi.localIP().toString(), 3000);
            previousMillis = now;
            lastSettingsFetch = now;
        }
    }

    // =====
    // Sensor reading & local control (works in BOTH modes)
    // =====
    if (now - lastSensorReadTime >= 1000) {
        lastSensorReadTime = now;
        readAllSensors();
        runDecisionLogic();
    }

    // =====
    // Online-only tasks

```

```

// =====
if (isOnlineMode) {
    if (now - previousMillis >= (unsigned long)uploadInterval * 1000) {
        previousMillis = now;
        sendSensorData();
    }

    if (now - lastSettingsFetch >= 2000) {
        lastSettingsFetch = now;
        fetchServerSettings();
    }
}

// =====
// Always-running tasks
// =====
updateBuzzer();

static unsigned long lastScreenUpdate = 0;
if (now - lastScreenUpdate >= 1000) {
    lastScreenUpdate = now;
    refreshOledScreen();
}
}

// =====
// Read Sensors
// =====
void readAllSensors() {
    float t = dhtSensor.readTemperature();
    float h = dhtSensor.readHumidity();

    // Update values and track min/max
    if (!isnan(t)) {
        temperature = t;
        if (!statsReady || isnan(minTemp) || t < minTemp) minTemp = t;
        if (!statsReady || isnan(maxTemp) || t > maxTemp) maxTemp = t;
    }
    if (!isnan(h)) {
        humidity = h;
        if (!statsReady || isnan(minHum) || h < minHum) minHum = h;
        if (!statsReady || isnan(maxHum) || h > maxHum) maxHum = h;
    }
    if (!isnan(t) || !isnan(h)) statsReady = true;

    digitalWrite(PIN_WATER_LEVEL_POWER, HIGH);
    delay(200);
    waterLevel = analogRead(PIN_WATER_LEVEL_SIGNAL);
    digitalWrite(PIN_WATER_LEVEL_POWER, LOW);

    waterHeightCm = SENSOR_BOTTOM_CM + ((float)waterLevel / 4095.0) * SENSOR_RANGE_CM;

    digitalWrite(PIN_RAIN_POWER, HIGH);
    delay(5);
    int rawAnalog = analogRead(PIN_RAIN_ANALOG);
    digitalWrite(PIN_RAIN_POWER, LOW);

    if (rawAnalog <= RAIN_INVALID_MAX) {
        rainAnalogValue = 4095;
        isRaining = false;
    } else {
        rainAnalogValue = rawAnalog;
        bool rawRain = (rainAnalogValue < RAIN_WET_THRESHOLD);
        static unsigned long rainSince = 0;
        static unsigned long drySince = 0;

```

```

        if (rawRain) {
            drySince = 0;
            if (rainSince == 0) rainSince = millis();
            else if (millis() - rainSince >= RAIN_DEBOUNCE_MS) isRaining = true;
        } else {
            rainSince = 0;
            if (drySince == 0) drySince = millis();
            else if (millis() - drySince >= 10000) isRaining = false;
        }
    }

    isTouched = (digitalRead(PIN_TOUCH) == HIGH);

    Serial.printf("RoomT:%.1fC RoomH:%.1f%% Water:%.1fcm (raw:%d) Rain:%s Touch:%s\n",
        temperature, humidity, waterHeightCm, waterLevel,
        isRaining ? "YES" : "No", isTouched ? "YES" : "No");
}

// =====
// Decision Logic
// =====

void runDecisionLogic() {
    // Drain cap
    bool waterTooHigh = (waterHeightCm > WATER_THRESHOLD_CM);
    bool shouldOpenDrain = waterTooHigh || drainOpenOverride;
    if (shouldOpenDrain && !isDrainOpen()) {
        openDrain();
        if (!drainOpenOverride) {
            showTemporaryMessage("Water HIGH\nDraining...", 3000);
            queueBuzzer(3000, 200, 150, 3);
        }
        Serial.println("Drain OPENED.");
    } else if (!shouldOpenDrain && isDrainOpen()) {
        closeDrain();
        Serial.println("Drain CLOSED.");
    }
}

// Touch handling (with long-press WiFi reset)
static bool lastTouchRaw = false;
static unsigned long touchHoldStart = 0;
bool touchRaw = isTouched;
bool resetTriggered = false;

if (touchRaw) {
    if (touchHoldStart == 0) touchHoldStart = millis();
    else if (millis() - touchHoldStart > 5000) {
        resetTriggered = true;
        showTemporaryMessage("Resetting WiFi...", 3000);
        queueBuzzer(2000, 300, 200, 3);
        delay(1000);
        wm.resetSettings();
        prefs.begin("smartagriculture", false);
        prefs.clear();
        prefs.end();
        ESP.restart();
    }
} else {
    touchHoldStart = 0;
}

if (!resetTriggered && touchRaw && !lastTouchRaw) {
    if (millis() - lastTouchToggleTime > TOUCH_COOLDOWN_MS) {
        lidManualOpen = !lidManualOpen;
        lastTouchToggleTime = millis();
        if (lidManualOpen) {

```



```

        showTemporaryMessage("Container\nOPEN (Manual)", 2000);
        queueBuzzer(1200, 150, 100, 1);
        Serial.println("Touch: lid MANUAL OPEN.");
    } else {
        showTemporaryMessage("Container\nCLOSED (Manual)", 2000);
        queueBuzzer(800, 150, 100, 1);
        Serial.println("Touch: lid MANUAL CLOSE.");
    }
}

lastTouchRaw = touchRaw;

// Rain buzzer
if (isRaining && !lastRainState) {
    if (!rainBuzzerFired) {
        queueBuzzer(2500, 400, 100, 1);
        rainBuzzerFired = true;
        showTemporaryMessage("Rain detected!\nOpening lid", 3000);
        Serial.println("Rain STARTED → buzz once + open lid.");
    }
}
if (!isRaining && lastRainState) {
    rainBuzzerFired = false;
    showTemporaryMessage("Rain stopped", 2000);
    Serial.println("Rain STOPPED → reset buzzer arm.");
}
lastRainState = isRaining;

// Environmental lid control
bool tooHumid = (!isnan(humidity) && humidity > humidityThreshold);
bool tooHot = (!isnan(temperature) && temperature > tempThreshold);

bool desiredOpen;
if (lidOpenOverride) {
    desiredOpen = true;
} else if (isRaining) {
    desiredOpen = true;
} else if (tooHot) {
    desiredOpen = true;
} else if (lidManualOpen) {
    desiredOpen = true;
} else {
    desiredOpen = false;
}

if (desiredOpen && !isLidOpen()) {
    openLid();
    Serial.println("Lid OPENED.");
} else if (!desiredOpen && isLidOpen()) {
    closeLid();
    Serial.println("Lid CLOSED.");
}
}

// =====
// Send Data to Server
// =====
void sendSensorData() {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("WiFi not connected. Skip upload.");
        return;
    }

    bool tooHumid = (!isnan(humidity) && humidity > humidityThreshold);
    bool tooHot = (!isnan(temperature) && temperature > tempThreshold);

```

```

String status = "Normal";
if (waterHeightCm > WATER_THRESHOLD_CM) {
    status = "Danger";
} else if (isRaining || tooHot || tooHumid) {
    status = "Warning";
}

String url = serverHost + "/php/api_post_data.php";
Serial.print("Upload → "); Serial.println(url);

HTTPClient http;
http.begin(url);
http.addHeader("Content-Type", "application/json");

String json = "{";
json += "\"device_id\": \"" + deviceID + "\", ";
json += "\"temperature\": " + String(temperature, 1) + ", ";
json += "\"humidity\": " + String(humidity, 1) + ", ";
json += "\"water_level\": " + String(waterLevel) + ", ";
json += "\"water_drop\": " + String(isRaining ? 1 : 0) + ", ";
if (!isnan(minTemp)) {
    json += "\"min_temp\": " + String(minTemp, 1) + ", ";
    json += "\"max_temp\": " + String(maxTemp, 1) + ", ";
}
if (!isnan(minHum)) {
    json += "\"min_hum\": " + String(minHum, 1) + ", ";
    json += "\"max_hum\": " + String(maxHum, 1) + ", ";
}
json += "\"status\": \"" + status + "\"";
json += "}";

int code = http.POST(json);
Serial.printf("HTTP %d, Water: %.1f cm (raw:%d)\n", code, waterHeightCm, waterLevel);
http.end();
}

// =====
// Fetch Settings from Server
// =====

void fetchServerSettings() {
    if (WiFi.status() != WL_CONNECTED) return;

    String url = serverHost + "/php/api_get_settings.php?device_id=" + deviceID;
    Serial.print("Sync settings → "); Serial.println(url);

    HTTPClient http;
    http.begin(url);

    int code = http.GET();
    if (code > 0) {
        String response = http.getString();
        tempThreshold = extractFloat(response, "temp_threshold", tempThreshold);
        humidityThreshold = extractFloat(response, "humidity_threshold", humidityThreshold);
        waterThreshold = extractInt(response, "water_level_threshold", waterThreshold);
        uploadInterval = extractInt(response, "upload_interval", uploadInterval);
        buzzerEnabled = extractInt(response, "buzzer_enabled", buzzerEnabled ?
0) == 1;
        drainOpenOverride = extractInt(response, "servo2_open", drainOpenOverri
1 : 0) == 1;
        lidOpenOverride = extractInt(response, "servo1_open", lidOpenOverride
1 : 0) == 1;

        Serial.println("Settings synced:");
    }
}

```

```

        Serial.printf("  TempThr:%.1f  HumThr:%.1f  WaterThr:%d  Int:%d  Buzzer:%s  LidOv:%d\n",
        DrainOv:%s\n",
                        tempThreshold, humidityThreshold, waterThreshold, uploadInterval,
                        buzzerEnabled ? "ON" : "OFF",
                        lidOpenOverride ? "ON" : "OFF",
                        drainOpenOverride ? "ON" : "OFF");
    } else {
        Serial.printf("Settings fetch failed: %d\n", code);
    }
    http.end();
}

// =====
// JSON Extraction Helpers
// =====
String extractValue(const String& json, const String& key) {
    String search = "\"" + key + "\"";
    int idx = json.indexOf(search);
    if (idx == -1) return "";
    idx += search.length();
    while (idx < (int)json.length() && json[idx] == ' ') idx++;
    int end = idx;
    while (end < (int)json.length() && json[end] != ',' && json[end] != '}') end++;
    String val = json.substring(idx, end);
    val.trim();
    if (val.startsWith("\"") && val.endsWith("\""))
        val = val.substring(1, val.length() - 1);
    return val;
}

float extractFloat(const String& json, const String& key, float defaultVal) {
    String v = extractValue(json, key);
    return v.length() ? v.toFloat() : defaultVal;
}

int extractInt(const String& json, const String& key, int defaultVal) {
    String v = extractValue(json, key);
    return v.length() ? v.toInt() : defaultVal;
}

```